EX. No: 1

Implement the functions using Math library

Date: 24.06.2026

Aim:

To write a Python program to perform mathematical operations using the Math library.

Code:

```
import math
print("1. Square Root\n2. Power\n3. Factorial\n4. Ceiling\n5. Floor\n6. Absolute Value\n7. GCD\n8. Sine\n9. Cosine\n10. Logarithm\n0. Exit")
while True:
    try:
        choice = int(input("Enter choice: "))
        match choice:
            case 0:
                break
            case 1:
                value = float(input("Enter a non-negative number: "))
                if value < 0:
                    print("Square root requires a non-negative number.")
                else: print("Square Root:", math.sqrt(value))
            case 2:
                first = float(input("Enter base: "))
                second = float(input("Enter exponent: "))
                print("Power:", math.pow(first, second))
            case 3:
                value = int(input("Enter a non-negative integer: "))
                if value < 0:
                    print("Factorial is not defined for negative integers.")
                else:
                    print("Factorial:", math.factorial(value))
            case 4:
                value = float(input("Enter a number: "))
                print("Ceiling:", math.ceil(value))
            case 5:
```

```
            value = float(input("Enter a number: "))
            print("Floor:", math.floor(value))
        case 6:
            value = float(input("Enter a number: "))
            print("Absolute value:", math.fabs(value))
        case 7:
            first = int(input("Enter first integer: "))
            second = int(input("Enter second integer: "))
            print("GCD:", math.gcd(first, second))
        case 8:
            angle = float(input("Enter angle in degrees: "))
            print("Sine:", math.sin(math.radians(angle)))
        case 9:
            angle = float(input("Enter angle in degrees: "))
            print("Cosine:", math.cos(math.radians(angle)))
        case 10:
            value = float(input("Enter a positive number: "))
            if value <= 0:
                print("Logarithm requires a positive number.")
            else: print("Logarithm:", math.log(value))
        case _:
            print("Invalid choice. Select a listed option.")
except (ValueError) as error:
    print("Invalid value:", error)
```

Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\01_math_library.py
1. Square Root
2. Power
3. Factorial
4. Ceiling
5. Floor
6. Absolute Value
7. GCD
8. Sine
9. Cosine
10. Logarithm
0. Exit
Enter choice: 1
Enter a non-negative number: 4
Square Root: 2.0
Enter choice: 2
Enter base: 2
Enter exponent: 4
Power: 16.0
Enter choice: 3
Enter a non-negative integer: 6
Factorial: 720
Enter choice: 5
Enter a number: 6.5
Floor: 6
Enter choice: 6
Enter a number: -80
Absolute value: 80.0
Enter choice: 0
PS H:\Projects\LabProjects\ML_Programs>
```

Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 2

Create and manipulate multidimensional arrays using NumPy

Date: 30.06.2026

Aim:

To write a Python program to create and manipulate multidimensional arrays using the NumPy library.

Code:

```
import numpy as np
try:
    ro = int(input("Enter the No of Rows: "))
    co = int(input("Enter the No of Columns: "))
    if ro <= 0 or co <= 0:
        raise ValueError("Rows and columns must be positive.")
    arr = np.zeros((ro, co), dtype=int)
    def cre(arr, ro, co):
        for i in range(ro):
            for j in range(co):
                arr[i, j] = int(input(f"Enter the Number [{i},{j}]: "))
        return arr
    def PrintArr(arr):
        rows, cols = arr.shape
        for i in range(rows):
            for j in range(cols):
                print(arr[i, j], end="  ")
            print()
    arr = cre(arr, ro, co)
    while True:
        try:
            print("\nPress 1 for Display\nPress 2 for Transpose\nPress 3 for Min and Max\nPress 4 for Sum\nPress 5 for Row and Column Sum\nPress 6 for Diagonal Values\nPress 7 for Update\nPress 8 for Array Details\nPress 9 for Sub Array\nPress 10 for Matrix Addition and Subtraction\nPress 0 to Exit")
            ch = int(input("Enter your Choice: "))
            match ch:
                case 1:
```

```
        print("Array Elements are:")
        PrintArr(arr)
    case 2:
        print("Transpose Array:")
        PrintArr(arr.T)
    case 3:
        print(f"Minimum Element: {np.min(arr)}")
        print(f"Maximum Element: {np.max(arr)}")
    case 4:
        print(f"Sum of Array: {np.sum(arr)}")
    case 5:
        print(f"Sum of Rows: {np.sum(arr, axis=1)}")
        print(f"Sum of Columns: {np.sum(arr, axis=0)}")
    case 6:
        if ro == co:
            print(f"Diagonal Values: {np.diag(arr)}")
        else:
            print("Row and column sizes are not the same.")
    case 7:
        r = int(input("Update Row Index: "))
        c = int(input("Update Column Index: "))

        if 0 <= r < ro and 0 <= c < co:
            arr[r, c] = int(input("Enter New Element: "))
            print("Element updated successfully!")
        else:
            print("Index out of bounds!")
    case 8:
        print(f"Array Size: {np.size(arr)}")
        print(f"Array Dimensions: {np.ndim(arr)}")
        print(f"Array Shape: {np.shape(arr)}")
    case 9:
        print(f"Row indices: 0 to {ro - 1}, Column indices: 0 to {co - 1}")
        rs = int(input("Sub Array Row Start Index: "))
        re = int(input("Sub Array Row End Index: "))
        cs = int(input("Sub Array Column Start Index: "))
```

```
                ce = int(input("Sub Array Column End Index: "))

                if 0 <= rs <= re < ro and 0 <= cs <= ce < co:
                    subarray = arr[rs:re + 1, cs:ce + 1]
                    print("Extracted Sub Array:")
                    PrintArr(subarray)
                else:
                    print("Invalid indices!")
            case 10:
                print("Enter Elements for Matrix 2:")
                arr2 = cre(np.zeros((ro, co), dtype=int), ro, co)
                print("Matrix Addition Result:")
                PrintArr(arr + arr2)
                print("Matrix Subtraction Result:")
                PrintArr(arr - arr2)
            case 0:
                print("Exiting program. Goodbye!")
                break
            case _:
                print("Invalid choice! Please try again.")
        except (ValueError, IndexError) as e:
            print("Invalid input:", e)
        except Exception as e:
            print("Error:", e)
except (ValueError, OverflowError) as e:
    print("Invalid input:", e)
except Exception as e:
    print("Error:", e)
```

Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\02_numpy_arrays.py
Enter the No of Rows: 2
Enter the No of Columns: 2
Enter the Number [0,0]: 1
Enter the Number [0,1]: 2
Enter the Number [1,0]: 3
Enter the Number [1,1]: 4

Press 1 for Display
Press 2 for Transpose
Press 3 for Min and Max
Press 4 for Sum
Press 5 for Row and Column Sum
Press 6 for Diagonal Values
Press 7 for Update
Press 8 for Array Details
Press 9 for Sub Array
Press 10 for Matrix Addition and Subtraction
Press 0 to Exit
Enter your Choice: 1
Array Elements are:
1  2
3  4
```

```

Press 1 for Display
Press 2 for Transpose
Press 3 for Min and Max
Press 4 for Sum
Press 5 for Row and Column Sum
Press 6 for Diagonal Values
Press 7 for Update
Press 8 for Array Details
Press 9 for Sub Array
Press 10 for Matrix Addition and Subtraction
Press 0 to Exit
Enter your Choice: 3
Minimum Element: 1
Maximum Element: 4

Press 1 for Display
Press 2 for Transpose
Press 3 for Min and Max
Press 4 for Sum
Press 5 for Row and Column Sum
Press 6 for Diagonal Values
Press 7 for Update
Press 8 for Array Details
Press 9 for Sub Array
Press 10 for Matrix Addition and Subtraction
Press 0 to Exit
Enter your Choice: 4
Sum of Array: 10
```

Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 3

Perform statistical functions using Statistics library

Date: 04.07.2026

Aim:

To write a Python program to perform statistical operations using the Statistics library.

Code:

```
import statistics as st
print("\nPress 1 for Mean()\nPress 2 for Median()\nPress 3 for Mode()\nPress 4 for Standard Deviation\nPress 5 for Variance\nPress 6 for Harmonic Mean\nPress 7 for Median Low\nPress 8 for Median High\nPress 9 for Population Standard Deviation\nPress 10 for Population Variance\nPress 0 to Exit")
while True:
    try:
        choice = int(input("Enter your choice: "))
        match choice:
            case 0:
                print("Program terminated.")
                break
            case 1:
                data = list(map(float, input("Enter numbers separated by spaces: ").split()))
                print("Mean:", st.mean(data))
            case 2:
                data = list(map(float, input("Enter numbers separated by spaces: ").split()))
                print("Median:", st.median(data))
            case 3:
                data = list(map(float, input("Enter numbers separated by spaces: ").split()))
                print("Mode:", st.mode(data))
            case 4:
                data = list(map(float, input("Enter numbers separated by spaces: ").split()))
                print("Standard Deviation:", st.stdev(data))
            case 5:
                data = list(map(float, input("Enter numbers separated by spaces: ").split()))
                print("Variance:", st.variance(data))
            case 6:
                data = list(map(float, input("Enter numbers separated by spaces: ").split()))
```

```
            print("Harmonic Mean:", st.harmonic_mean(data))
        case 7:
            data = list(map(float, input("Enter numbers separated by spaces: ").split()))
            print("Median Low:", st.median_low(data))
        case 8:
            data = list(map(float, input("Enter numbers separated by spaces: ").split()))
            print("Median High:", st.median_high(data))
        case 9:
            data = list(map(float, input("Enter numbers separated by spaces: ").split()))
            print("Population Standard Deviation:", st.pstdev(data))
        case 10:
            data = list(map(float, input("Enter numbers separated by spaces: ").split()))
            print("Population Variance:", st.pvariance(data))
        case _:
            print("Invalid choice!")
    except Exception as e:
        print("Error:", e)
```

Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\03_statistics_library.py

Press 1 for Mean()
Press 2 for Median()
Press 3 for Mode()
Press 4 for Standard Deviation
Press 5 for Variance
Press 6 for Harmonic Mean
Press 7 for Median Low
Press 8 for Median High
Press 9 for Population Standard Deviation
Press 10 for Population Variance
Press 0 to Exit
Enter your choice: 1
Enter numbers separated by spaces: 1 2 3 4 5
Mean: 3.0
Enter your choice: 2
Enter numbers separated by spaces: 1 2 3 4 5
Median: 3.0
Enter your choice: 3
Enter numbers separated by spaces: 1 2 2 3 1 2
Mode: 2.0
Enter your choice: 4
Enter numbers separated by spaces: 12 23 1 24
Standard Deviation: 10.801234497346433
Enter your choice: 5
Enter numbers separated by spaces: 5 1 2 1
Variance: 3.5833333333333335
Enter your choice: 0
Program terminated.
```

Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 4

DataFrame implementation using Pandas

Date: 09.07.2026

Aim:

To write a Python program to create and manipulate DataFrames using the Pandas library.

Code:

```
import pandas as p
import numpy as np
print("\n1. Head Row\n2. Tail Row\n3. Specific Head and Tail Row\n4. Column Selection\n5. Add New Column\n6. Filter Data\n7. DataFrame Information\n8. Fillna\n9. Dropna\n10. IsNull and NotNull\n11. Replace\n12. Exit")
def Main():
    n = int(input("Enter the number of Series: "))
    name = []
    episodes = []
    year = []
    for i in range(n):
        print(f"---------- Enter Series {i+1} Details ----------")
        na = input("Enter Series Name: ")
        ep = int(input("Enter Episode Count: "))
        yr = int(input("Enter Release Year: "))

        name.append(na)
        episodes.append(ep)
        year.append(yr)
    data = {
        "Series Name": name,
        "Episode Count": episodes,
        "Release Year": year
    }
    df = p.DataFrame(data)
    print("\n---------- DataFrame ----------")
    print(df)
    Implement(df)
```

```
def Implement(df):
    while True:
        try:
            ch = int(input("Enter Your Choice: "))
            match ch:
                case 1:
                    print("Head Row:\n", df.head())
                case 2:
                    print("Tail Row:\n", df.tail())
                case 3:
                    h = int(input("Enter the Number of Rows: "))
                    print("Head Row:\n", df.head(h))
                    print("Tail Row:\n", df.tail(h))
                case 4:
                    print("All Columns:\n", df.columns)
                    s = input("Enter Specific Column: ")
                    print(df[s])
                case 5:
                    ne = input("Enter the New Column Name: ")
                    da = []
                    for i in range(len(df)):
                        data = input("Enter the Data: ")
                        da.append(data)
                    df[ne] = da
                    print("New DataFrame:\n", df)
                case 6:
                    ma = int(input("Enter the Minimum Episode Count: "))
                    print("Filtered Data:\n", df[df["Episode Count"] > ma])
                case 7:
                    print("DataFrame Information:")
                    df.info()
                    print("DataFrame Description:\n", df.describe())
                    print("DataFrame Shape:", df.shape)
                case 8:
                    print("Fillna:")
```

```
                df["Episode Count"] = df["Episode Count"].fillna(df["Episode Count"].mean())
                print(df["Episode Count"])
            case 9:
                print("Dropna:")
                print(df["Episode Count"].dropna())
            case 10:
                print("IsNull:")
                print(df.isnull())
                print("NotNull:")
                print(df.notnull())
            case 11:
                print("Replace:")
                print(df["Episode Count"].replace(np.nan, 0))
            case 12:
                break
            case _:
                print("Invalid Choice!")
        except Exception as e:
            print("Error:", e)
try:
    Main()
except Exception as e:
    print("Error:", e)
```

Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\04_pandas_dataframe.py

1. Head Row
2. Tail Row
3. Specific Head and Tail Row
4. Column Selection
5. Add New Column
6. Filter Data
7. DataFrame Information
8. Fillna
9. Dropna
10. IsNull and NotNull
11. Replace
12. Exit
Enter the number of Series: 3
---------- Enter Series 1 Details ----------
Enter Series Name: Supernatural
Enter Episode Count: 327
Enter Release Year: 2005
---------- Enter Series 2 Details ----------
Enter Series Name: Grimm
Enter Episode Count: 123
Enter Release Year: 2011
---------- Enter Series 3 Details ----------
Enter Series Name: The Originals
Enter Episode Count: 92
Enter Release Year: 2013
```

```
------------- DataFrame ---------------
     Series Name  Episode Count  Release Year
0   Supernatural            327          2005
1          Grimm            123          2011
2  The Originals             92          2013
Enter Your Choice: 6
Enter the Minimum Episode Count: 120
Filtered Data:
     Series Name  Episode Count  Release Year
0  Supernatural            327          2005
1         Grimm            123          2011
Enter Your Choice: 7
DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Series Name    3 non-null      str
 1   Episode Count  3 non-null      int64
 2   Release Year   3 non-null      int64
dtypes: int64(2), str(1)
memory usage: 204.0 bytes
DataFrame Description:
        Episode Count  Release Year
count       3.000000      3.000000
mean      180.666667   2009.666667
std       127.672759      4.163332
min        92.000000   2005.000000
25%       107.500000   2008.000000
50%       123.000000   2011.000000
```

Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 5

Handling the missing values in a dataset

Date: 13.07.2026

Aim:

To write a Python program to identify and handle missing values in a dataset using Pandas.

Code:

```
import pandas as p
import numpy as np
print("\n1. Fillna (Mean)\n2. DropNa\n3. IsNull and NotNull\n4. Replace Value\n5. Fillna (Median)\n6. Exit")
def CSV():
    df = p.read_csv("datasets/handle_missing.csv")
    Main(df)
def Main(df):
    while True:
        try:
            ch = int(input("Enter Your Choice: "))
            match ch:
                case 1:
                    print("---------- FillNa (Mean) ----------")
                    df["Episode Count"] = round(df["Episode Count"].fillna(df["Episode Count"].mean()), 2)
                    print("Mean To Fill Value:\n", df)
                case 2:
                    print("---------- DropNa ----------")
                    print(df["Episode Count"].dropna())
                case 3:
                    print("---------- IsNull ----------")
                    print(df.isnull())
                    print("---------- NotNull ----------")
                    print(df.notnull())
                case 4:
                    print("---------- Replace ----------")
                    value = int(input("Enter the Replace Value: "))
                    print(df["Episode Count"].replace(np.nan, value))
```

```
            case 5:
                print("-------------- FillNa (Median) --------------")
                df["Episode Count"] = round(df["Episode Count"].fillna(df["Episode Count"].median()), 2)
                print("Median To Fill Value:\n", df)
            case 6:
                print("Program terminated.")
                break
            case _:
                print("Invalid Choice!")
        except Exception as e:
            print("Error:", e)
try:
    CSV()
except Exception as e:
    print("Error:", e)
```

Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\05_missing_values.py

1. Fillna (Mean)
2. DropNa
3. IsNull and NotNull
4. Replace Value
5. Fillna (Median)
6. Exit
Enter Your Choice: 1
-------------- FillNa (Mean) --------------
Mean To Fill Value:
             Series Name  Episode Count  Release Year
0         Supernatural          327.0        2005.0
1                Grimm          123.0        2011.0
2            Teen Wolf          140.5        2011.0
3  The Vampire Diaries          171.0           NaN
4        The Originals           92.0        2013.0
5        Sleepy Hollow           62.0        2013.0
6        Shadowhunters          140.5        2016.0
7             Legacies           68.0           NaN
```

```
Enter Your Choice: 3
-------------- IsNull --------------
   Series Name  Episode Count  Release Year
0        False          False         False
1        False          False         False
2        False           True         False
3        False          False          True
4        False          False         False
5        False          False         False
6        False           True         False
7        False          False          True
-------------- NotNull --------------
   Series Name  Episode Count  Release Year
0         True           True          True
1         True           True          True
2         True          False          True
3         True           True         False
4         True           True          True
5         True           True          True
6         True          False          True
7         True           True         False
```

```
Enter Your Choice: 4
-------------- Replace --------------
Enter the Replace Value: 0
0    327.0
1    123.0
2      0.0
3    171.0
4     92.0
5     62.0
6      0.0
7     68.0
Name: Episode Count, dtype: float64
Enter Your Choice: 6
Program terminated.
```

Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 6

Load a dataset and perform the Data analysis

Date: 16.07.2026

Aim:

To write a Python program to load a dataset and perform data analysis using Pandas.

Code:

```
import pandas as pd
import numpy as np
data = pd.read_csv("datasets/zomato.csv")
# see the columns and type
print(data.dtypes)
# remove unwanted columns
data.drop(columns=["reviews_list"],inplace=True)
# drop rows where name is not present
data["name"].dropna()
# fill na values
data["votes"].fillna(0)
data["rate"].fillna("0/5")
for column in data.columns:
    if column not in ["votes"]:
        data[column].fillna("")
# remove duplicates
def getWithoutDuplicates(d):
    return d.drop_duplicates(subset="name",keep="first")
# convert rate to float
def changeRate(d):
    if d in ["NEW","-"]:
        return 0
    rate = str(d).split("/")[0]
    return float(rate)
data["rate"] = data["rate"].apply(changeRate)
data["rate"] = pd.to_numeric(data["rate"],errors="coerce")
# Important ColumnsimpColumns =
["address","name","online_order","book_table","rate","location","cuisines","rest_type","dish_liked"]
```

```
gropuedColumns = ["name","rate"]
# Data analysis
# 1. Top Rated resturaents in each location
grouped_data = data.groupby("location")
for group in grouped_data.groups:
    print(f"\tLocation: {group}")
    grouped = grouped_data.get_group(group)
    sorted_data = grouped.sort_values(by="rate",ascending=False,na_position="last")
    sorted_data = getWithoutDuplicates(sorted_data)
    sorted_data = sorted_data.head(3)
    print(sorted_data[gropuedColumns])
# 2 Count How many online and offline orders
totalCount = len(data)
onlineCount = 0
offlineCount = 0
def countOrder(d):
    if d.lower() == "yes":
        global onlineCount
        onlineCount += 1
    else:
        global offlineCount
        offlineCount += 1
data["online_order"].apply(countOrder)
print(f"Online Orders: {onlineCount}/{totalCount}")
print(f"Offline Orders: {offlineCount}/{totalCount}")
#3 Total Resturants and total by resturants type
resturantsData = getWithoutDuplicates(data)
print(f"Total Resturants: {len(resturantsData)}")
groupByRestType = resturantsData.groupby("rest_type")
for restGroup in groupByRestType.groups:
    currentGroupCount = groupByRestType.get_group(restGroup)["rest_type"].count()
    print(f"Resturant Type: {restGroup} = {currentGroupCount}")
#4 Average rating by restaurant type
avg_rating = resturantsData.groupby("rest_type")["rate"].mean().sort_values(ascending=False)
print("Average Rating by Restaurant Type:")
print(avg_rating.head(10))
```

Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\06_load_and_analyze_dataset.py
url                            str
address                        str
name                           str
online_order                   str
book_table                     str
rate                           str
votes                        int64
phone                          str
location                       str
rest_type                      str
dish_liked                     str
cuisines                       str
approx_cost(for two people)    str
reviews_list                   str
menu_item                      str
listed_in(type)                str
listed_in(city)                str
dtype: object
```

```
Online Orders: 30444/51717
Offline Orders: 21273/51717
Total Resturants: 8792
Resturant Type: Bakery = 197
Resturant Type: Bakery, Beverage Shop = 3
Resturant Type: Bakery, Cafe = 7
Resturant Type: Bakery, Dessert Parlor = 20
Resturant Type: Bakery, Quick Bites = 32
Resturant Type: Bakery, Sweet Shop = 1
Resturant Type: Bar = 87
Resturant Type: Bar, Cafe = 1
Resturant Type: Bar, Casual Dining = 51
Resturant Type: Bar, Lounge = 2
Resturant Type: Bar, Pub = 1
Resturant Type: Bar, Quick Bites = 1
Resturant Type: Beverage Shop = 140
Resturant Type: Beverage Shop, Cafe = 1
Resturant Type: Beverage Shop, Dessert Parlor = 6
Resturant Type: Beverage Shop, Quick Bites = 37
Resturant Type: Bhojanalya = 1
Resturant Type: Cafe = 415
Resturant Type: Cafe, Bakery = 15
Resturant Type: Cafe, Bar = 1
Resturant Type: Cafe, Casual Dining = 18
Resturant Type: Cafe, Dessert Parlor = 16
Resturant Type: Cafe, Lounge = 1
```

```
Average Rating by Restaurant Type:
rest_type
Pub, Cafe                      4.700000
Bar, Pub                       4.600000
Microbrewery, Pub              4.487500
Pub, Microbrewery              4.416667
Cafe, Lounge                   4.400000
Microbrewery, Bar              4.400000
Casual Dining, Irani Cafee     4.400000
Fine Dining, Lounge            4.400000
Fine Dining, Microbrewery      4.300000
Fine Dining, Bar               4.275000
Name: rate, dtype: float64
PS H:\Projects\LabProjects\ML_Programs>
```

Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 7

Load a dataset and display its structure with Statistical summaries using Pandas

Date: 21.07.2026

Aim:

To write a Python program to load a dataset and display its structure and statistical summaries using Pandas.

Code:

```
import pandas as pd
data = pd.read_csv("datasets/DataSet.csv")
print("\n1. Display Dataset Structure\n2. Display Statistical Summary\n3. Particular Column Summary\n4. Group Characters by Series\n5. Group Characters by Character Type\n6. Group by First Appearance Year\n7. Series and Character Type\n8. Exit")
while True:
    try:
        ch = int(input("Enter Your Choice: "))
        match ch:
            case 1:
                print("\n\tDataset Structure")
                print(data)
                print("\n\tFirst Five Characters")
                print(data.head())
                print("\n\tLast Five Characters")
                print(data.tail())
                print("\nDataset Shape:", data.shape)
                print("\nColumn Names:", data.columns)
                print("\nData Types:\n", data.dtypes)
                print("\nDataset Information:")
                data.info()
                print("\nMissing Values:\n", data.isnull().sum())
            case 2:
                print("\n\tStatistical Summary")
                print(data.describe(include="all"))
            case 3:
                print("\nAvailable Columns:\n", data.columns)
```

```
            col = input("Enter the Column Name: ")
            print("\n\tColumn Summary")
            print(data[col].describe())
        case 4:
            print("\n\tCharacters Grouped by Series")
            print(data.groupby("Series")["Character Name"].apply(list))
        case 5:
            print("\n\tCharacters Grouped by Character Type")
            print(data.groupby("Character Type")["Character Name"].apply(list))
        case 6:
            print("\n\tCharacters by First Appearance Year")
            print(data.groupby("First Appearance Year")["Character Name"].apply(list))
        case 7:
            print("\n\tSeries and Character Type")
            print(data.groupby(["Series", "Character Type"]).size())
        case 8:
            print("Program terminated.")
            break
        case _: print("Invalid Choice!")
    except Exception as e:
        print("Error:", e)
```

Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\07_pandas_structure_summary.py

1. Display Dataset Structure
2. Display Statistical Summary
3. Particular Column Summary
4. Group Characters by Series
5. Group Characters by Character Type
6. Group by First Appearance Year
7. Series and Character Type
8. Exit
Enter Your Choice: 2

        Statistical Summary
        Character Name        Series Character Type  First Appearance Year Species / Role
count               16            16             16              16.000000             16
unique              16             8             10                    NaN              9
top     Sam Winchester  Supernatural        Vampire                    NaN          Human
freq                 1             4              4                    NaN              6
mean               NaN           NaN            NaN            2010.312500            NaN
std                NaN           NaN            NaN               3.380705            NaN
min                NaN           NaN            NaN            2005.000000            NaN
25%                NaN           NaN            NaN            2009.000000            NaN
50%                NaN           NaN            NaN            2010.000000            NaN
75%                NaN           NaN            NaN            2011.000000            NaN
max                NaN           NaN            NaN            2018.000000            NaN
```

Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 8

Load a dataset and display its structure with graphs using Pandas and Matplotlib

Date: 25.07.2026

Aim:

To write a Python program to load a dataset and visualize its structure using Pandas and Matplotlib.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
data = pd.read_csv("datasets/DataSet.csv")
print("\n1. Display Dataset Structure\n2. Display Statistical Summary\n3. Characters by Series (Bar Graph)\n4. Characters by Character Type (Bar Graph)\n5. First Appearance Year (Line Graph)\n6. Series Distribution (Pie Chart)\n7. Exit")
while True:
    try:
        ch = int(input("Enter Your Choice: "))
        match ch:
            case 1:
                print("\n\tDataset Structure")
                print(data)
                print("\n\tFirst Five Characters")
                print(data.head())
                print("\n\tLast Five Characters")
                print(data.tail())
                print("\nDataset Shape:", data.shape)
                print("\nColumn Names:", data.columns)
                print("\nData Types:\n", data.dtypes)
                print("\nDataset Information:")
                data.info()
                print("\nMissing Values:\n", data.isnull().sum())
            case 2:
                print("\n\tStatistical Summary")
                print(data.describe(include="all"))
            case 3:
                print("\n\tCharacters by Series")
```

```
        counts = data["Series"].value_counts()
        print(counts)
        counts.plot(kind="bar", title="Characters by Series")
        plt.xlabel("Series")
        plt.ylabel("Number of Characters")
        plt.xticks(rotation=45, ha="right")
        plt.tight_layout()
        plt.show()
    case 4:
        print("\n\tCharacters by Character Type")
        counts = data["Character Type"].value_counts()
        print(counts)
        counts.plot(kind="bar", title="Characters by Type")
        plt.xlabel("Character Type")
        plt.ylabel("Number of Characters")
        plt.xticks(rotation=45, ha="right")
        plt.tight_layout()
        plt.show()
    case 5:
        print("\n\tCharacters by First Appearance Year")
        counts = data["First Appearance Year"].value_counts().sort_index()
        print(counts)
        counts.plot(kind="line", marker="o", title="Characters by First Appearance Year")
        plt.xlabel("First Appearance Year")
        plt.ylabel("Number of Characters")
        plt.grid(True)
        plt.tight_layout()
        plt.show()
    case 6:
        print("\n\tSeries Distribution")
        counts = data["Series"].value_counts()
        print(counts)
        counts.plot(kind="pie", autopct="%1.1f%%", title="Series Distribution")
        plt.ylabel("")
        plt.tight_layout()
        plt.show()
```

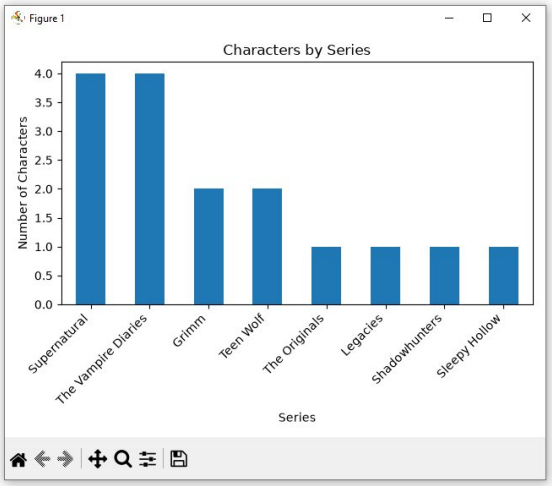
```
        case 7:
            print("Program terminated.")
            break
        case _:
            print("Invalid Choice!")
    except Exception as e:
        print("Error:", e)
```

Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\08_pandas_matplotlib_graphs.py

1. Display Dataset Structure
2. Display Statistical Summary
3. Characters by Series (Bar Graph)
4. Characters by Character Type (Bar Graph)
5. First Appearance Year (Line Graph)
6. Series Distribution (Pie Chart)
7. Exit
Enter Your Choice: 3

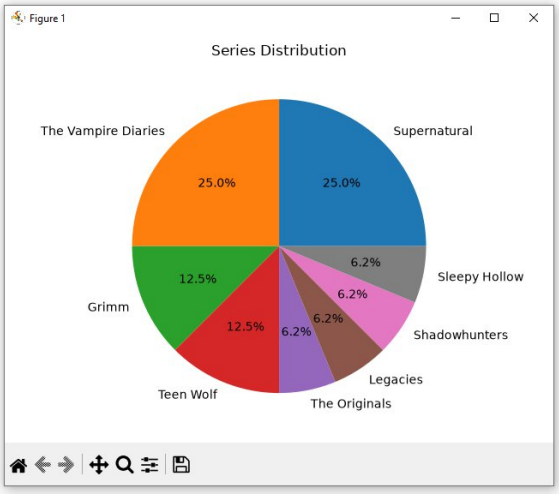
        Characters by Series
Series
Supernatural           4
The Vampire Diaries    4
Grimm                  2
Teen Wolf              2
The Originals          1
Legacies               1
Shadowhunters          1
Sleepy Hollow          1
Name: count, dtype: int64
```




```
Enter Your Choice: 3

        Characters by Series
Series
Supernatural           4
The Vampire Diaries    4
Grimm                  2
Teen Wolf              2
The Originals          1
Legacies               1
Shadowhunters          1
Sleepy Hollow          1
Name: count, dtype: int64
Enter Your Choice: 6

        Series Distribution
Series
Supernatural           4
The Vampire Diaries    4
Grimm                  2
Teen Wolf              2
The Originals          1
Legacies               1
Shadowhunters          1
Sleepy Hollow          1
Name: count, dtype: int64
```



Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 9

Perform Visualization using Seaborn

Date: 30.07.2026

Aim:

To write a Python program to visualize a dataset using the Seaborn library.

Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
data = pd.read_csv("datasets/Ben10Dataset.csv")
print("\n1. Display Dataset\n2. Scatter Plot\n3. Histogram\n4. Box Plot\n5. Heatmap\n6. Violin Plot\n7. Count Plot\n8. Pair Plot\n0. Exit")
while True:
    try:
        ch = int(input("Enter Your Choice: "))
        match ch:
            case 1:
                print("\n---------- Ben 10 Alien Dataset ----------")
                print(data.head(5))
            case 2:
                print("\n---------- Scatter Plot ----------")
                sns.scatterplot(data=data, x="Height (m)", y="Power Level", hue="Series")
                plt.title("Alien Height vs Power Level")
                plt.show()
            case 3:
                print("\n---------- Histogram ----------")
                sns.histplot(data=data, x="Power Level", bins=8, kde=True)
                plt.title("Distribution of Alien Power Levels")
                plt.show()
            case 4:
                print("\n---------- Box Plot ----------")
                sns.boxplot(data=data, x="Series", y="Height (m)")
                plt.title("Alien Height by Series")
                plt.xticks(rotation=30)
```

```
            plt.tight_layout()
            plt.show()
        case 5:
            print("\n---------- Heatmap ----------")
            corr = data[["Power Level", "Height (m)", "First Appearance Year"]].corr()
            sns.heatmap(corr, annot=True, cmap="coolwarm")
            plt.title("Alien Dataset Correlation Heatmap")
            plt.show()
        case 6:
            print("\n---------- Violin Plot ----------")
            sns.violinplot(data=data, x="Series", y="Power Level")
            plt.title("Power Level Distribution by Series")
            plt.xticks(rotation=30)
            plt.tight_layout()
            plt.show()
        case 7:
            print("\n---------- Count Plot ----------")
            sns.countplot(data=data, x="Element")
            plt.title("Number of Aliens by Element")
            plt.xticks(rotation=45)
            plt.tight_layout()
            plt.show()
        case 8:
            print("\n---------- Pair Plot ----------")
            sns.pairplot(data, vars=["Power Level", "Height (m)", "First Appearance Year"], hue="Series")
            plt.show()
        case 0:
            print("Program terminated.")
            break
        case _:
            print("Invalid Choice!")
except Exception as e:
    print("Error:", e)
```

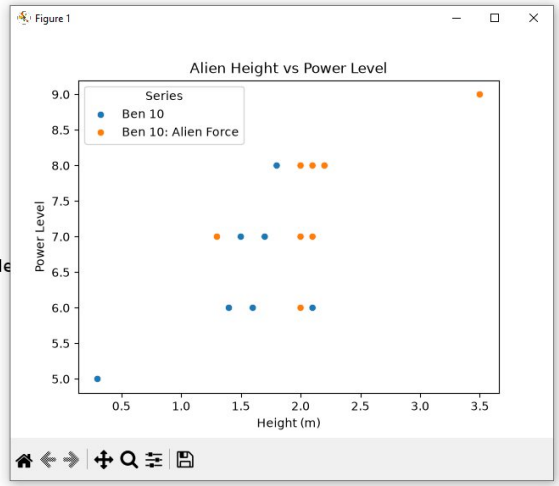
Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\09_seaborn_visualization.py

1. Display Dataset
2. Scatter Plot
3. Histogram
4. Box Plot
5. Heatmap
6. Violin Plot
7. Count Plot
8. Pair Plot
0. Exit
Enter Your Choice: 1

---------- Ben 10 Alien Dataset ----------
    Alien Name  Series   Element  Power Level  He
0    Four Arms  Ben 10  Strength            8
1    Heatblast  Ben 10      Fire            8
2         XLR8  Ben 10     Speed            7
3  Diamondhead  Ben 10   Crystal            8
4     Wildmutt  Ben 10     Beast            6
Enter Your Choice: 2

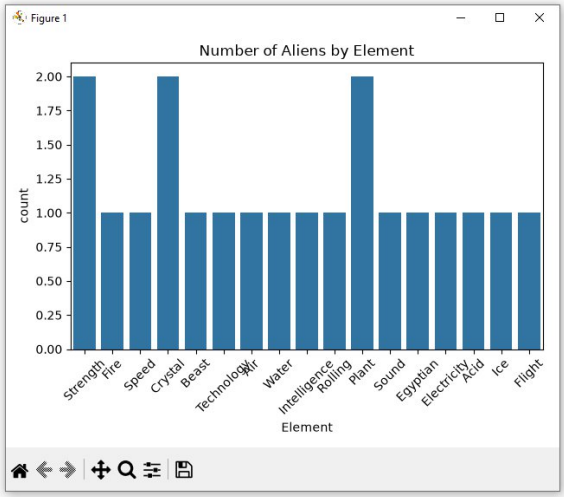
---------- Scatter Plot ----------
```




```
PS H:\Projects\LabProjects\ML_Programs> python .\09_seaborn_visualization.py

1. Display Dataset
2. Scatter Plot
3. Histogram
4. Box Plot
5. Heatmap
6. Violin Plot
7. Count Plot
8. Pair Plot
0. Exit
Enter Your Choice: 7

---------- Count Plot ----------
```



Result:

Thus, the dataset is visualized successfully using the Seaborn library.

EX. No: 10

Implementation of Covariance and Correlation

Date: 03.08.2026

Aim:

To write a Python program to calculate covariance and correlation between variables in a dataset.

Code:

```
def covariance(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    total = 0
    for i in range(n):
        total += (x[i] - mean_x) * (y[i] - mean_y)
    return total / (n - 1)
def correlation(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    numerator = 0
    sum_x = 0
    sum_y = 0
    for i in range(n):
        dx = x[i] - mean_x
        dy = y[i] - mean_y
        numerator += dx * dy
        sum_x += dx ** 2
        sum_y += dy ** 2
    denominator = (sum_x * sum_y) ** 0.5
    if denominator == 0:
        return "Correlation cannot be calculated"
    return numerator / denominator
n = int(input("Enter number of data points: "))
x = []
y = []
```

```
for i in range(n):
    x.append(float(input("Enter X value: ")))
    y.append(float(input("Enter Y value: ")))
print("\n--- MENU ---")
print("1. Display Data")
print("2. Covariance")
print("3. Correlation")
print("0. Exit")
while True:
    ch = int(input("Enter your choice: "))
    if ch == 1:
        print("X =", x)
        print("Y =", y)
    elif ch == 2:
        if n < 2:
            print("Enter at least 2 data points")
        else:
            print("Covariance =", covariance(x, y))
    elif ch == 3:
        if n < 2:
            print("Enter at least 2 data points")
        else:
            print("Correlation =", correlation(x, y))
    elif ch == 0:
        print("Program terminated.")
        break
    else:
        print("Invalid choice!")
```

Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\10_covariance_correlation.py
Enter number of data points: 5
Enter X value: 1
Enter Y value: 2
Enter X value: 2
Enter Y value: 4
Enter X value: 3
Enter Y value: 6
Enter X value: 4
Enter Y value: 8
Enter X value: 5
Enter Y value: 10

--- MENU ---
1. Display Data
2. Covariance
3. Correlation
0. Exit
Enter your choice: 1
X = [1.0, 2.0, 3.0, 4.0, 5.0]
Y = [2.0, 4.0, 6.0, 8.0, 10.0]
Enter your choice: 2
Covariance = 5.0
Enter your choice: 3
Correlation = 1.0
Enter your choice: 0
Program terminated.
```

Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 11

Perform Normalization in a given dataset

Date: 06.08.2026

Aim:

To write a Python program to normalize the values in a given dataset using appropriate normalization techniques.

Code:

```
import pandas as pd
import numpy as np
from scipy import stats
from sklearn.preprocessing import MinMaxScaler

df = pd.read_csv("datasets/Ben10Dataset.csv")
def MinMax(val, min, max, nmin, nmax):
    if min == max:
        return "Normalization cannot be calculated."
    result = (((val - min) / (max - min)) * (nmax - nmin)) + nmin
    return f"{val} ====> {np.round(result, 2)}"
def ZScore(val, mean, std):
    if std == 0:
        return "Z-Score cannot be calculated."
    result = (val - mean) / std
    return f"{val} ====> {round(result, 2)}"
def decimal(val):
    dig = len(str(abs(int(val))))
    return val / 10**dig
num = df.describe().columns.to_list()
print("\n1. MinMax Normalization\n2. Z-Score Normalization\n3. Decimal Scale Normalization\n4. Exit")
while True:
    try:
        ch = int(input("Enter Your Choice: "))
        match ch:
            case 1:
                print("\n---------- MinMax Normalization ----------")
```

```
            for index, col in enumerate(num):
                print(f"{index} => {col}", end="  ")
            colindex = int(input("\nEnter Column Index: "))
            min = df[num[colindex]].head(10).min()
            max = df[num[colindex]].head(10).max()
            print("Minimum:", min, "Maximum:", max)
            val = float(input("Enter the Value: "))
            nmin = float(input("Enter the New Min Value: "))
            nmax = float(input("Enter the New Max Value: "))
            print(MinMax(val, min, max, nmin, nmax))
        case 2:
            print("\n---------- Z-Score Normalization ----------")
            for index, col in enumerate(num):
                print(f"{index} => {col}", end="  ")
            colindex = int(input("\nEnter Column Index: "))
            mean = df[num[colindex]].head(10).mean()
            std = df[num[colindex]].head(10).std()
            print("Mean:", round(mean, 2), "Standard Deviation:", round(std, 2))
            val = float(input("Enter the Value: "))
            print(ZScore(val, mean, std))
        case 3:
            print("\n---------- Decimal Scale Normalization ----------")

            val = int(input("Enter the Number: "))

            print("Normalized Value:", decimal(val))
        case 4:
            print("Exit Program")
            break
        case _:
            print("Invalid Choice!")
except Exception as e:
    print("Error:", e)
```

Output:

```
1. MinMax Normalization
2. Z-Score Normalization
3. Decimal Scale Normalization
4. Exit
Enter Your Choice: 1

---------- MinMax Normalization ----------
0 => Power Level  1 => Height (m)  2 => First Appearance Year
Enter Column Index: 0
Minimum: 5 Maximum: 8
Enter the Value: 8
Enter the New Min Value: 0
Enter the New Max Value: 1
8.0 ====> 1.0
Enter Your Choice: 2

---------- Z-Score Normalization ----------
0 => Power Level  1 => Height (m)  2 => First Appearance Year
Enter Column Index: 0
Mean: 6.8 Standard Deviation: 1.03
Enter the Value: 8
8.0 ====> 1.16
Enter Your Choice: 3

---------- Decimal Scale Normalization ----------
Enter the Number: 250
Normalized Value: 0.25
```

Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 12

Perform feature selection using Extra Trees Classifier

Date: 10.08.2026

Aim:

To write a Python program to perform feature selection using the Extra Trees Classifier algorithm.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.ensemble import ExtraTreesClassifier
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target
model = ExtraTreesClassifier(n_estimators=100, random_state=42)
model.fit(X, y)
importance = model.feature_importances_
feature_importance = pd.DataFrame({
   "Feature": X.columns,
   "Importance": importance
})
feature_importance = feature_importance.sort_values(
   by="Importance",
   ascending=False
)
print("\n1. Display Dataset\n2. Feature Importance\n3. Plot Feature Importance\n4. Predict Iris Species\n0. Exit")
while True:
   try:
      ch = int(input("Enter Your Choice: "))
      match ch:
         case 1:
            print("\n---------- Iris Dataset ----------")
            print(X.head(10))
            print("\nTarget Species:")
```

```
            print(iris.target_names)
        case 2:
            print("\n---------- Feature Importance ----------")
            print(feature_importance)
        case 3:
            print("\n---------- Feature Importance Graph ----------")
            plt.figure(figsize=(8, 5))
            plt.bar(
                feature_importance["Feature"],
                feature_importance["Importance"]
            )
            plt.xticks(rotation=45)
            plt.xlabel("Features")
            plt.ylabel("Importance")
            plt.title("Feature Selection using Extra Trees Classifier")
            plt.tight_layout()
            plt.show()
        case 4:
            print("\n---------- Predict Iris Species ----------")
            sl = float(input("Enter Sepal Length (cm): "))
            sw = float(input("Enter Sepal Width (cm): "))
            pl = float(input("Enter Petal Length (cm): "))
            pw = float(input("Enter Petal Width (cm): "))
            new_data = pd.DataFrame(
                [[sl, sw, pl, pw]],
                columns=iris.feature_names
            )
            prediction = model.predict(new_data)
            print("Predicted Iris Species:", iris.target_names[prediction[0]])
        case 0:
            print("Program terminated.")
            break
        case _:
            print("Invalid Choice!")
    except Exception as e:
        print("Error:", e)
```

Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\12_extra_trees_feature_selection.py

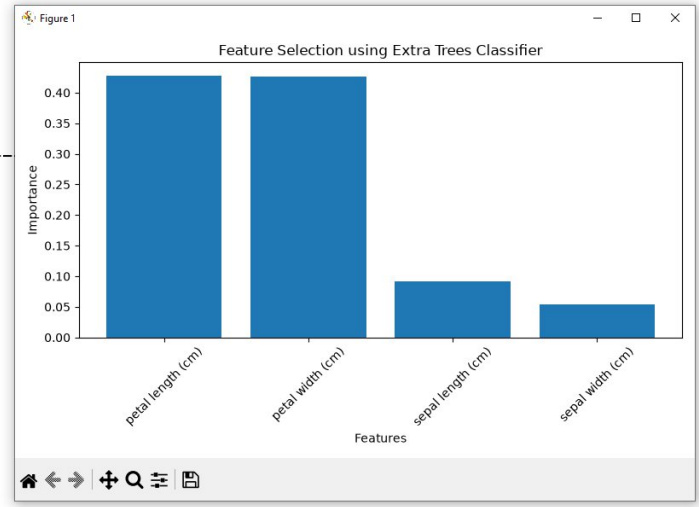
1. Display Dataset
2. Feature Importance
3. Plot Feature Importance
4. Predict Iris Species
0. Exit
Enter Your Choice: 2

---------- Feature Importance ----------
             Feature  Importance
2  petal length (cm)    0.428177
3   petal width (cm)    0.425765
0  sepal length (cm)    0.091767
1   sepal width (cm)    0.054292
Enter Your Choice: 4

---------- Predict Iris Species ----------
Enter Sepal Length (cm): 6
Enter Sepal Width (cm): 2.9
Enter Petal Length (cm): 4.5
Enter Petal Width (cm): 1.5
Predicted Iris Species: versicolor
Enter Your Choice: 0
Program terminated.
```

```
1. Display Dataset
2. Feature Importance
3. Plot Feature Importance
4. Predict Iris Species
0. Exit
Enter Your Choice: 3

---------- Feature Importance Graph ----
```



Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 13

Analyze the dataset using Julius AI

Date: 17.08.2026

Aim:

To analyze a given dataset using Julius AI and interpret the results.

Steps:

Step 1: Open Julius AI

Open Google Chrome, visit https://julius.ai/, and sign up or log in to your account.

Step 2: Upload the Dataset

Open a new chat or data-analysis workspace and upload Supernatural_Series_Characters.csv.

Step 3: Understand the Dataset

Enter the following prompt:

Analyze the uploaded dataset and display the number of rows, columns, column names, data types, first five rows, and missing values in each column.

Step 4: Perform Statistical Analysis

Enter the following prompt:

Calculate the mean, median, mode, minimum, maximum, variance, and standard deviation for the numerical columns, especially First Appearance Year and Power Rating.

Step 5: Create Visualizations

Enter the following prompt:

Generate a bar chart showing the number of characters by series, a pie chart showing species distribution, a histogram of character power ratings, a box plot of power ratings by series, and a scatter plot of First Appearance Year versus Power Rating.

Step 6: Analyze Patterns and Relationships

Enter the following prompt:

Calculate the correlation between First Appearance Year and Power Rating. Identify the highest and lowest power ratings, missing values, possible outliers, and five important findings from the dataset.

Step 7: Clean and Export the Dataset

Enter the following prompt:

Check for duplicate rows, handle missing values appropriately, identify possible outliers, and create a cleaned dataset without modifying the original. Export the cleaned dataset as a CSV file if supported.

Step 8: Generate the Final Report

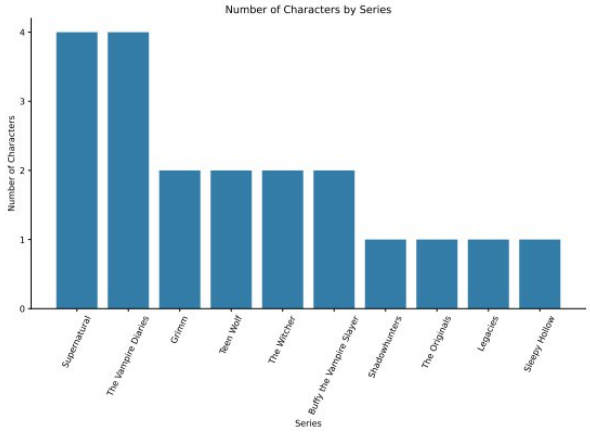
Enter the following prompt:

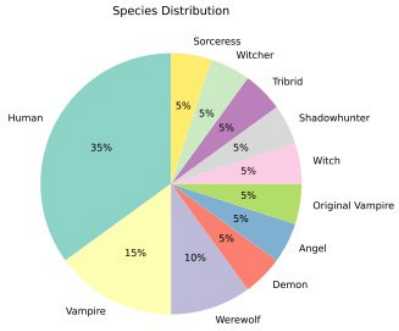
Prepare a dataset analysis report containing dataset information, statistical analysis, missing values, visualizations, correlation analysis, key findings, and a conclusion. Use the actual uploaded data and do not treat the sample power ratings as official rankings.

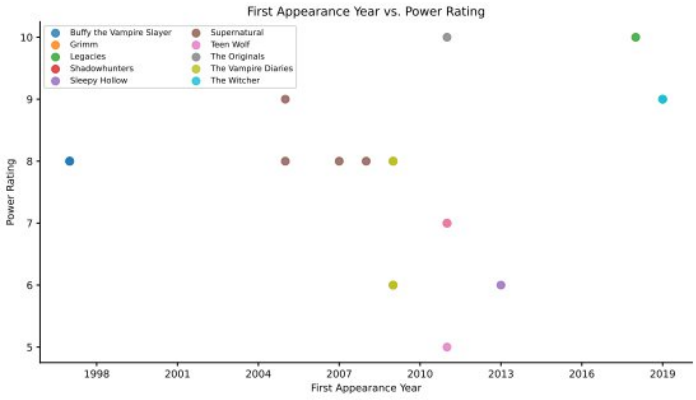
Step 9: Review and Download

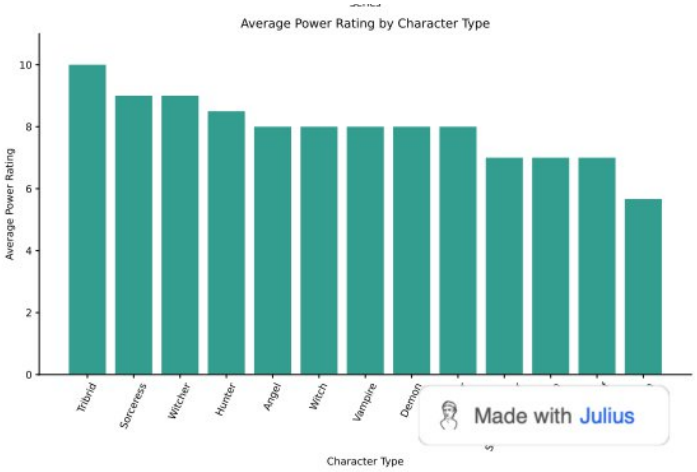
Review the analysis and download the report, charts, or cleaned dataset using the available export options.

Output:









Result:

Thus, the dataset is analyzed successfully using Julius AI, and the results are verified.

EX. No: 14

Analyze the dataset with dashboard using Bricks AI

Date: 22.08.2026

Aim:

To analyze a given dataset and create an interactive dashboard using Bricks AI.

Steps:

Step 1: Open Bricks AI

Open Google Chrome, visit https://www.thebricks.com/, and sign up or log in to your account.

Step 2: Create a New Project

Open a new workspace or create a new project in Bricks AI.

Step 3: Upload the Dataset

Upload the billing and payment dataset in CSV or Excel format, or import it from a supported spreadsheet source.

Step 4: Analyze the Dataset

Enter the following prompt:

Analyze the uploaded dataset. Display the number of rows, columns, column names, data types, and a summary of the billing and payment data.

Step 5: Perform Statistical Analysis

Enter the following prompt:

Analyze the 2026 billing and payment data. Calculate total sales, total payments, average bill value, customer counts, payment-mode distribution, and other relevant billing KPIs.

Step 6: Create the Dashboard

Enter the following prompt:

Create an interactive dashboard using the uploaded dataset. Include KPI cards, total records, billing and payment summaries, daily revenue trends, top customers and vehicles, payment-mode distribution, collection timing, underpayments, and advance usage.

Step 7: Add Charts and Filters

Enter the following prompt:

Create suitable charts to show daily revenue trends, top customers and vehicles, payment-mode distribution, and billing versus payment activity. Add suitable filters or slicers where supported.

Step 8: Identify Important Insights

Enter the following prompt:

Explain the important insights from the dashboard in simple English. Identify sales trends, customer concentration, highest and lowest values, collection delays, underpayments, advance usage, outliers, and data quality issues where the dataset supports these findings.

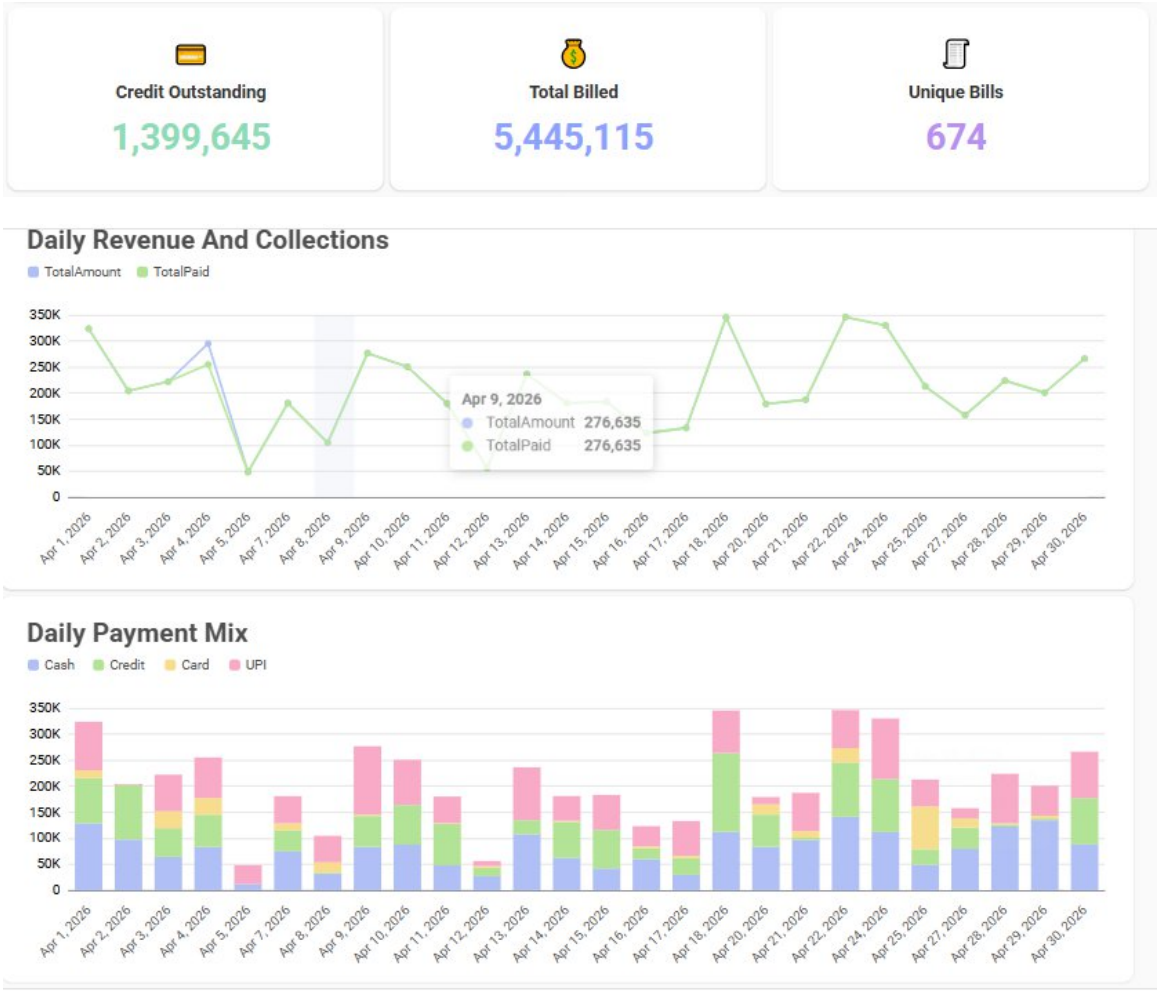
Step 9: Customize and Verify the Dashboard

Review the dashboard and adjust the layout, chart types, titles, labels, and formatting. Verify that the calculations and visualizations correctly represent the uploaded data.

Step 10: Save and Export the Dashboard

Save the dashboard and export or share it using the available options in Bricks AI.

Output:



Result:

Thus, the dataset is analyzed and the dashboard is created successfully using Bricks AI.

EX. No: 15

Perform feature extraction using Principal Component Analysis

Date: 27.08.2026

Aim:

To write a Python program to perform feature extraction using Principal Component Analysis (PCA).

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
iris = load_iris()
X = iris.data
y = iris.target
print("Feature Names:")
print(iris.feature_names)
print("\nDataset Shape:", X.shape)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)
print("\nTotal Variance Explained:",
    sum(pca.explained_variance_ratio_))
print("\nFirst 5 PCA-transformed samples:")
print(X_pca[:5])
plt.figure(figsize=(8, 6))
for i, target_name in enumerate(iris.target_names):
  plt.scatter(
    X_pca[y == i, 0],
    X_pca[y == i, 1],
    label=target_name
  )
```

```
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA of Iris Dataset")
plt.legend()
plt.grid()
plt.show()
```

Output:

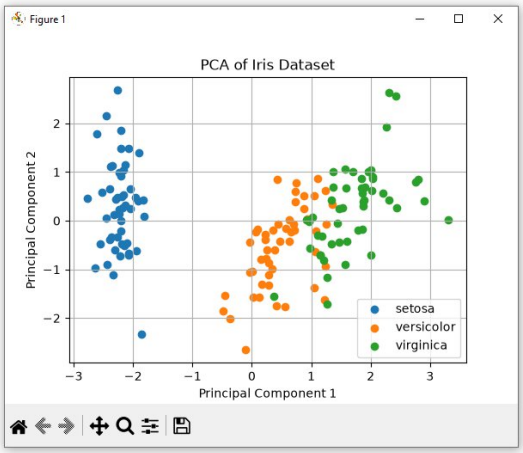
```
PS H:\Projects\LabProjects\ML_Programs> python .\15_pca_feature_extraction.py
Feature Names:
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

Dataset Shape: (150, 4)

Explained Variance Ratio:
[0.72962445 0.22850762]

Total Variance Explained: 0.9581320720000166

First 5 PCA-transformed samples:
[[-2.26470281  0.4800266 ]
 [-2.08096115 -0.67413356]
 [-2.36422905 -0.34190802]
 [-2.29938422 -0.59739451]
 [-2.38984217  0.64683538]]
```



Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 16

Implement KNN classification algorithm for an iris dataset using scikit-learn

Date: 01.09.2026

Aim:

To write a Python program to classify the Iris dataset using the K-Nearest Neighbors (KNN) algorithm with scikit-learn.

Code:

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
irisData = load_iris()
x = irisData.data
y = irisData.target
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=30
)
knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(x_train, y_train)
print("Iris Dataset Loaded Successfully")
print("Model Accuracy:", knn.score(x_test, y_test))
s1 = float(input("Enter sepal length: "))
s2 = float(input("Enter sepal width: "))
s3 = float(input("Enter petal length: "))
s4 = float(input("Enter petal width: "))
user_pre = knn.predict([[s1, s2, s3, s4]])
flower = irisData.target_names[user_pre[0]]
print("\nPredicted Flower Name:", flower)
print("\nPredictions on Test Data:")
print(knn.predict(x_test))
```

Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\16_knn_iris.py
Iris Dataset Loaded Successfully
Model Accuracy: 0.9666666666666667
Enter sepal length: 5.1
Enter sepal width: 3.5
Enter petal length: 1.4
Enter petal width: 0.2

Predicted Flower Name: setosa

Predictions on Test Data:
[0 0 0 2 1 1 2 2 1 2 0 2 1 1 0 1 0 0 0 1 2 0 0 0 2 2 2 2 0 1]
PS H:\Projects\LabProjects\ML_Programs>
```

Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 17

Home Price Prediction using Linear Regression for error based learning

Date: 07.09.2026

Aim:

To write a Python program to predict house prices using Linear Regression and evaluate the prediction error.

Code:

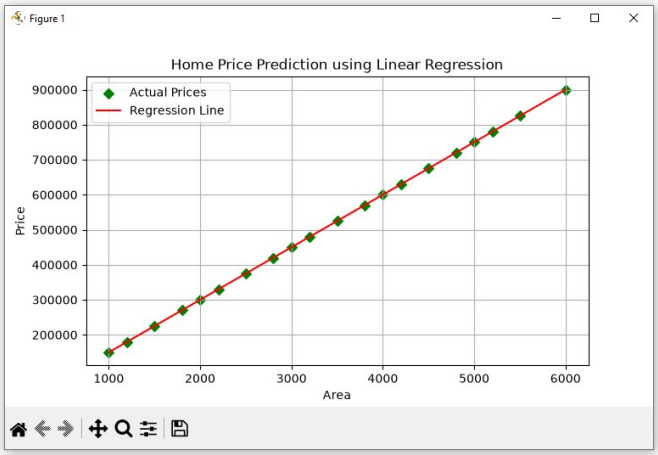
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import linear_model
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
# Create a local dataset
df = pd.DataFrame({
   'area': [1000, 1200, 1500, 1800, 2000,
            2200, 2500, 2800, 3000, 3200,
            3500, 3800, 4000, 4200, 4500,
            4800, 5000, 5200, 5500, 6000],
   'price': [150000, 180000, 225000, 270000, 300000,
             330000, 375000, 420000, 450000, 480000,
             525000, 570000, 600000, 630000, 675000,
             720000, 750000, 780000, 825000, 900000]
})
print("Home Price Dataset:")
print(df.head(5))
X = df[['area']]
y = df['price']
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)
reg = linear_model.LinearRegression()
reg.fit(X_train, y_train)
y_pred = reg.predict(X_test)
print("\nTraining samples:", len(X_train))
```

```
print("Testing samples:", len(X_test))
print("R2 score:", r2_score(y_test, y_pred))
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
# Plot 1: Regression Line
reg.fit(X, y)
plt.figure(figsize=(8, 5))
plt.scatter(df['area'], df['price'], color='green',
          marker='D', label='Actual Prices')
plt.plot(df['area'], reg.predict(X), color='red',
        label='Regression Line')
plt.title('Home Price Prediction using Linear Regression')
plt.xlabel('Area')
plt.ylabel('Price')
plt.legend()
plt.grid()
plt.show()
# Plot 2: Prediction Errors
errors = y_test - y_pred
plt.figure(figsize=(8, 5))
plt.scatter(y_pred, errors, color='purple')
plt.axhline(y=0, color='red', linestyle='--')
plt.title('Prediction Errors')
plt.xlabel('Predicted Price')
plt.ylabel('Error')
plt.grid()
plt.show()
# Predict prices for new areas
area_df = pd.DataFrame({
   'area': [1000, 1600, 2100, 3300, 5000, 6500]
})
area_df['prices'] = reg.predict(area_df[['area']])
print("\nPredicted Home Prices:")
print(area_df)
```

Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\17_home_price_linear_regression.py
Home Price Dataset:
   area   price
0  1000  150000
1  1200  180000
2  1500  225000
3  1800  270000
4  2000  300000

Training samples: 16
Testing samples: 4
R2 score: 1.0
Mean Squared Error: 3.8116482626443515e-21
```




```
Predicted Home Prices:
   area     prices
0  1000  150000.0
1  1600  240000.0
2  2100  315000.0
3  3300  495000.0
4  5000  750000.0
5  6500  975000.0
PS H:\Projects\LabProjects\ML_Programs>
```

Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 18

Perform SVM Classification using scikit-learn

Date: 10.09.2026

Aim:

To write a Python program to perform classification using the Support Vector Machine (SVM) algorithm with scikit-learn.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, ConfusionMatrixDisplay)
# Load your local dataset
df = pd.read_csv("datasets/breast_cancer_dataset.csv")
print("Dataset shape:", df.shape)
print(df['diagnosis_label'].value_counts())
X = df.drop(columns=['diagnosis', 'diagnosis_label'])
y = df['diagnosis']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
svm_model = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
svm_model.fit(X_train_scaled, y_train)
y_pred = svm_model.predict(X_test_scaled)
acc = accuracy_score(y_test, y_pred)
print(f"\nAccuracy: {acc*100:.2f}%")
print("\nClassification Report:\n", classification_report(
```

```
    y_test, y_pred, target_names=['malignant', 'benign']))
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm, display_labels=['malignant', 'benign'])
disp.plot(cmap='Blues')
plt.title("SVM - Confusion Matrix (Breast Cancer Diagnosis)")
plt.tight_layout()
plt.savefig("svm_confusion_matrix.png", dpi=150)
plt.show()
svm_linear = SVC(kernel='linear', C=1.0, random_state=42)
svm_linear.fit(X_train_scaled, y_train)
acc_linear = accuracy_score(y_test, svm_linear.predict(X_test_scaled))
print(f"\nLinear kernel accuracy: {acc_linear*100:.2f}%")
print(f"RBF kernel accuracy   : {acc*100:.2f}%")
```

Output:

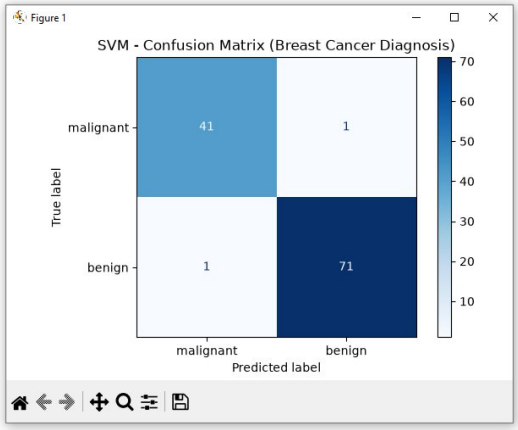
```
PS H:\Projects\LabProjects\ML_Programs> python .\18_svm_classification.py
Dataset shape: (569, 32)
diagnosis_label
benign       357
malignant    212
Name: count, dtype: int64

Accuracy: 98.25%

Classification Report:
              precision    recall  f1-score   support

   malignant       0.98      0.98      0.98        42
      benign       0.99      0.99      0.99        72

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```




```
Linear kernel accuracy: 97.37%
RBF kernel accuracy   : 98.25%
PS H:\Projects\LabProjects\ML_Programs>
```

Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 19

Implement Decision tree algorithm for an elective dataset

Date: 15.09.2026

Aim:

To write a Python program to implement the Decision Tree classification algorithm on a given dataset.

Code:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
# Load the given dataset
df = pd.read_ex("datasets/elective_dataset.xlsx")
X = df[["GENDER", "UG", "Cross Major"]]
y = df["ELECTIVE"]
X = pd.get_dummies(X, dtype=int)
print("\nEncoded Features:")
print(X)
feature_names = X.columns.tolist()
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.5,
    random_state=42,
    stratify=y
)
clf = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
acc = accuracy_score(y_test, y_pred)
print(f"Test Accuracy: {acc:.4f}")
print(classification_report(y_test, y_pred))
```

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("TEXT REPRESENTATION OF DECISION TREE")
print(export_text(
    clf,
    feature_names=feature_names
))
plt.figure(figsize=(15, 8))
plot_tree(
    clf,
    feature_names=feature_names,
    class_names=[str(c) for c in clf.classes_],
    filled=True,
    rounded=True
)
plt.title("Decision Tree - Elective Prediction")
plt.show()
```

Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\19_decision_tree_elective.py

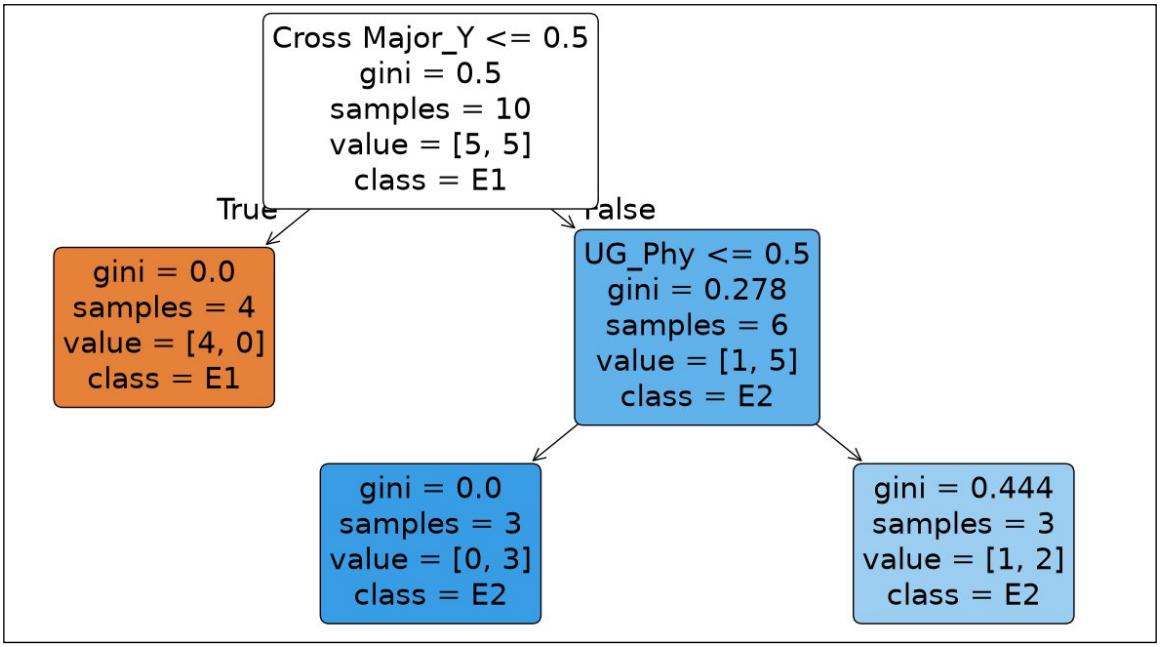
Encoded Features:
    GENDER_F  GENDER_M  UG_BCA  UG_Com  UG_Phy  Cross Major_N  Cross Major_Y
0          1         0       0       0       1              0              1
1          1         0       1       0       0              1              0
2          1         0       1       0       0              1              0
3          1         0       1       0       0              1              0
4          1         0       1       0       0              1              0
5          1         0       1       0       0              1              0
6          0         1       1       0       0              1              0
7          0         1       1       0       0              1              0
8          0         1       1       0       0              1              0
9          0         1       0       1       0              0              1
10         1         0       0       0       1              0              1
11         1         0       0       0       1              0              1
12         1         0       0       0       1              0              1
13         1         0       0       1       0              0              1
14         0         1       0       1       0              0              1
15         0         1       0       1       0              0              1
16         0         1       0       1       0              0              1
17         0         1       0       1       0              0              1
18         0         1       0       1       0              0              1
19         0         1       0       1       0              0              1
Test Accuracy: 0.9000
```

```
              precision    recall  f1-score   support

          E1       1.00      0.80      0.89         5
          E2       0.83      1.00      0.91         5

    accuracy                           0.90        10
   macro avg       0.92      0.90      0.90        10
weighted avg       0.92      0.90      0.90        10

Confusion Matrix:
[[4 1]
 [0 5]]
TEXT REPRESENTATION OF DECISION TREE
|--- Cross Major_Y <= 0.50
|   |--- class: E1
|--- Cross Major_Y >  0.50
|   |--- UG_Phy <= 0.50
|   |   |--- class: E2
|   |--- UG_Phy >  0.50
|   |   |--- class: E2
```



Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 20

Implement Random forest algorithm with customer churn dataset

Date: 19.09.2026

Aim:

To write a Python program to predict customer churn using the Random Forest classification algorithm.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, ConfusionMatrixDisplay)
df = pd.read_csv("datasets/customer_churn_dataset.csv")
print("Dataset shape:", df.shape)
print(df['Churn'].value_counts())
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df['TotalCharges'] = df['TotalCharges'].fillna(df['TotalCharges'].median())
df = df.drop(columns=['customerID'])
for col in df.select_dtypes(include=['object', 'str']).columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
X = df.drop(columns=['Churn'])
y = df['Churn']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=8,
    random_state=42,
    n_jobs=-1
)
```

```
rf_model.fit(X_train, y_train)
y_pred = rf_model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
print(f"\nAccuracy: {acc*100:.2f}%")
print("\nClassification Report:\n",
      classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Greens')
plt.title("Random Forest - Confusion Matrix")
plt.tight_layout()
plt.show()
importances = pd.Series(
    rf_model.feature_importances_, index=X.columns
).sort_values(ascending=False).head(10)
plt.figure(figsize=(8, 5))
importances.plot(kind='barh')
plt.gca().invert_yaxis()
plt.title("Top 10 Features Influencing Customer Churn")
plt.xlabel("Importance Score")
plt.tight_layout()
plt.show()
```

Output:

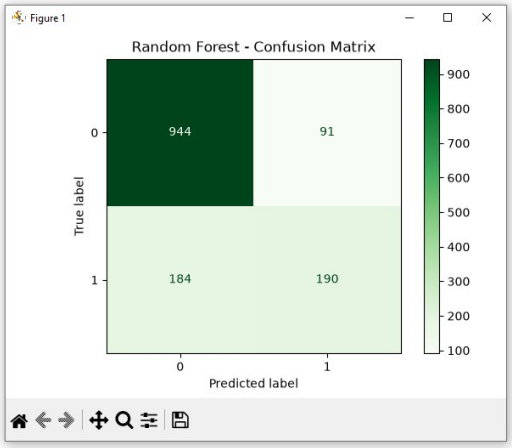
```
PS H:\Projects\LabProjects\ML_Programs> python .\20_random_forest_customer_churn.py
Dataset shape: (7043, 21)
Churn
No     5174
Yes    1869
Name: count, dtype: int64

Accuracy: 80.48%

Classification Report:
               precision    recall  f1-score   support

           0       0.84      0.91      0.87      1035
           1       0.68      0.51      0.58       374

    accuracy                           0.80      1409
   macro avg       0.76      0.71      0.73      1409
weighted avg       0.79      0.80      0.80      1409
```





Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 21

Perform Naïve Bayes Classification for loan dataset using scikit-learn

Date: 22.09.2026

Aim:

To write a Python program to perform loan approval classification using the Naïve Bayes algorithm with scikit-learn.

Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv('datasets/loan.csv')
print(df.head())
df.info()
sns.countplot(data=df, x='purpose', hue='not.fully.paid')
plt.xticks(rotation=45, ha='right')
plt.title("Loan Purpose vs Payment Status")
plt.tight_layout()
plt.show()
pre_df = pd.get_dummies(df, columns=['purpose'], drop_first=True)
print(pre_df.head())
from sklearn.model_selection import train_test_split
X = pre_df.drop('not.fully.paid', axis=1)
y = pre_df['not.fully.paid']
print(X)
print(y)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=125
)
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X_train, y_train)
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
```

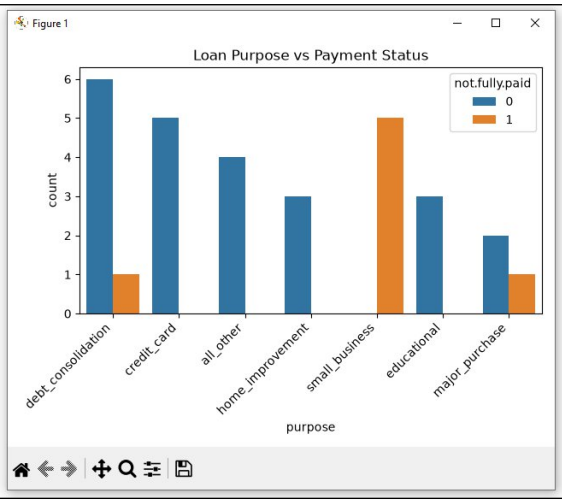
```
    ConfusionMatrixDisplay,
    f1_score,
    classification_report,
)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average="weighted")
print("\nAccuracy:", accuracy)
print("F1 Score:", f1)
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.title("Naive Bayes - Confusion Matrix")
plt.tight_layout()
plt.show()
```

Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\21_naive_bayes_loan.py
   credit.policy             purpose  int.rate  ...  delinq.2yrs  pub.rec  not.fully.paid
0              1  debt_consolidation    0.1189  ...            0        0               0
1              1         credit_card    0.1071  ...            0        0               0
2              1           all_other    0.1357  ...            1        0               0
3              1    home_improvement    0.1392  ...            0        0               0
4              0      small_business    0.1620  ...            2        1               1

[5 rows x 14 columns]
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
```

```
[5 rows x 14 columns]
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 14 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   credit.policy      30 non-null     int64
 1   purpose            30 non-null     str
 2   int.rate           30 non-null     float64
 3   installment        30 non-null     float64
 4   log.annual.inc     30 non-null     float64
 5   dti                30 non-null     float64
 6   fico               30 non-null     int64
 7   days.with.cr.line  30 non-null     int64
 8   revol.bal          30 non-null     int64
 9   revol.util         30 non-null     float64
 10  inq.last.6mths     30 non-null     int64
 11  delinq.2yrs        30 non-null     int64
 12  pub.rec            30 non-null     int64
 13  not.fully.paid     30 non-null     int64
dtypes: float64(5), int64(8), str(1)
memory usage: 3.4 KB
```



Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 22

Customer Segmentation based on Annual Income using K-means algorithm

Date: 28.09.2026

Aim:

To write a Python program to perform customer segmentation based on annual income using the K-means clustering algorithm.

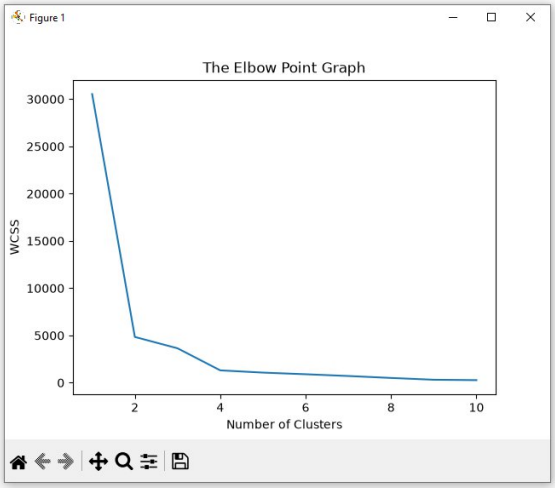
Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
customer_data = pd.read_csv("datasets/Mall_Customers.csv")
customer_data.head(25)
customer_data.shape
customer_data.info()
customer_data.isnull().sum()
X = customer_data.iloc[:,[3,4]].values
X[:25]
from sklearn.cluster import KMeans
wcss = []
for i in range(1,11):
    kmeans = KMeans(n_clusters=i, init = 'k-means++', random_state = 42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)
plt.plot(range(1,11), wcss)
plt.title('The Elbow Point Graph')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
kmeans = KMeans(n_clusters=5,init='k-means++',random_state=42)
y = kmeans.fit_predict(X)
print(y)
```

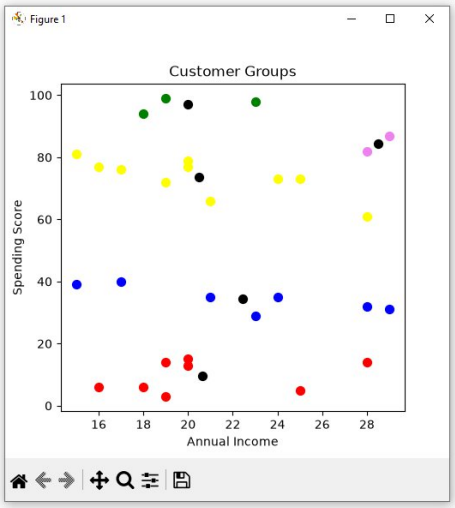
```
plt.figure(figsize=(6,6))
plt.scatter(X[y==0,0], X[y==0,1], s=50, c='green', label='Cluster 1')
plt.scatter(X[y==1,0], X[y==1,1], s=50, c='blue', label='Cluster 2')
plt.scatter(X[y==2,0], X[y==2,1], s=50, c='yellow', label='Cluster 3')
plt.scatter(X[y==3,0], X[y==3,1], s=50, c='red', label='Cluster 4')
plt.scatter(X[y==4,0], X[y==4,1], s=50, c='violet', label='Cluster 5')
plt.scatter(kmeans.cluster_centers_[:,0], kmeans.cluster_centers_[:,1], s=50, c='black', label='Centroids')
plt.title('Customer Groups')
plt.xlabel('Annual Income')
plt.ylabel('Spending Score')
plt.show()
```

Output:

```
PS H:\Projects\LabProjects\ML_Programs> python .\22_kmeans_customer_segmentation.py
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              30 non-null     int64
 1   Gender                  30 non-null     str
 2   Age                     30 non-null     int64
 3   Annual Income (k$)      30 non-null     int64
 4   Spending Score (1-100)  30 non-null     int64
dtypes: int64(4), str(1)
memory usage: 1.3 KB
```




```
PS H:\Projects\LabProjects\ML_Programs> python .\22_kmeans_customer_segmentation.py
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              30 non-null     int64
 1   Gender                  30 non-null     str
 2   Age                     30 non-null     int64
 3   Annual Income (k$)      30 non-null     int64
 4   Spending Score (1-100)  30 non-null     int64
dtypes: int64(4), str(1)
memory usage: 1.3 KB
[1 2 3 2 1 2 3 0 3 2 3 0 3 2 3 2 1 2 1 0 1 2 3 2 3 4 1 2 1 4]
```



Result:

Thus, the above code is executed and the output verified successfully.

EX. No: 23

Perform E-mail Classification using Levity

Date: 01.10.2026

Aim:

To classify emails into appropriate categories using the Levity AI platform.

Steps:

Step 1: Open Google Chrome on your computer.

Step 2: Visit the official website: https://levity.ai/

Step 3: Sign up or log in to Levity AI.

Step 4: Create a new AI workflow or classification project.

Step 5: Select the Text Classification option or a similar text-processing workflow, if available.

Step 6: Enter the project name as *Email Classification*.

Step 7: Prepare a dataset containing email messages and their corresponding categories.

Step 8: Create classification labels such as *Spam* and *Not Spam*.

Step 9: Upload the email dataset in a supported format, such as CSV, if supported.

Step 10: Map the email message column to the input field and the category column to the classification label.

Step 11: Review the dataset to ensure that the email messages and labels are correct.

Step 12: Train the AI model using the labeled email examples.

Step 13: Wait until the model training process is completed.

Step 14: Test the trained model using new email messages that were not included in the training dataset.

Step 15: Check whether the model correctly classifies each email as *Spam* or *Not Spam*.

Step 16: Review the model's performance and add more labeled examples if necessary.

Step 17: Configure an automated workflow to classify incoming emails, if email integration is supported by the available plan.

Step 18: Save the project and review the final classification results.

Output:

Result:

Thus, the email classification model is implemented and tested successfully using Levity AI.

EX. No: 24

Implementation of image classification using Teachable Machine

Date: 06.10.2026

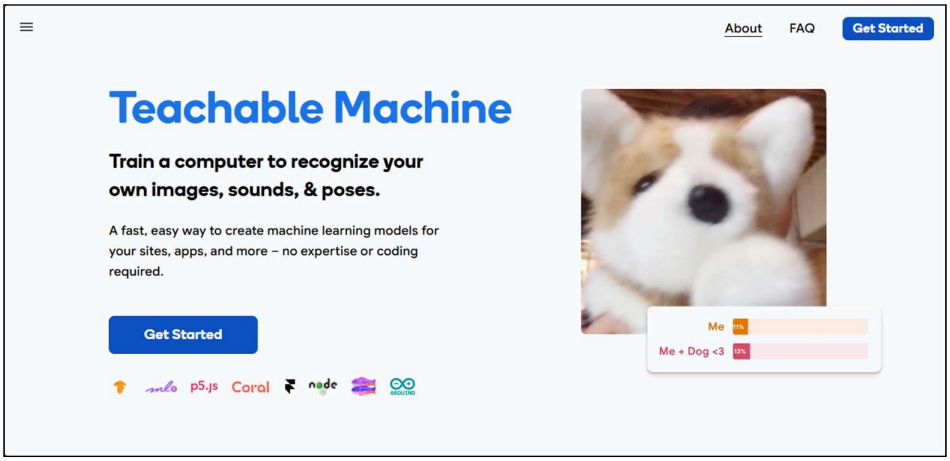
Aim:

To build and test an image classification model using Google's Teachable Machine.

Steps:

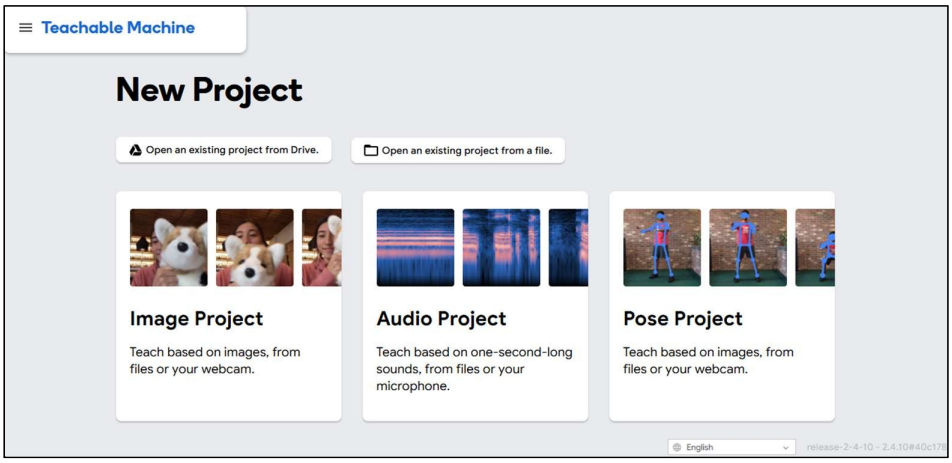
Step 1: Open google chrome to search teachable machine.

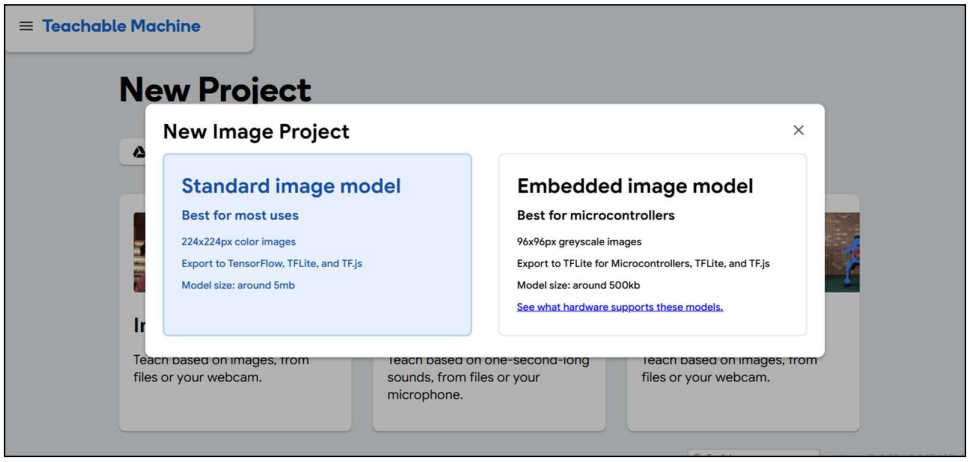
Step 2: Select the first link in the browser.



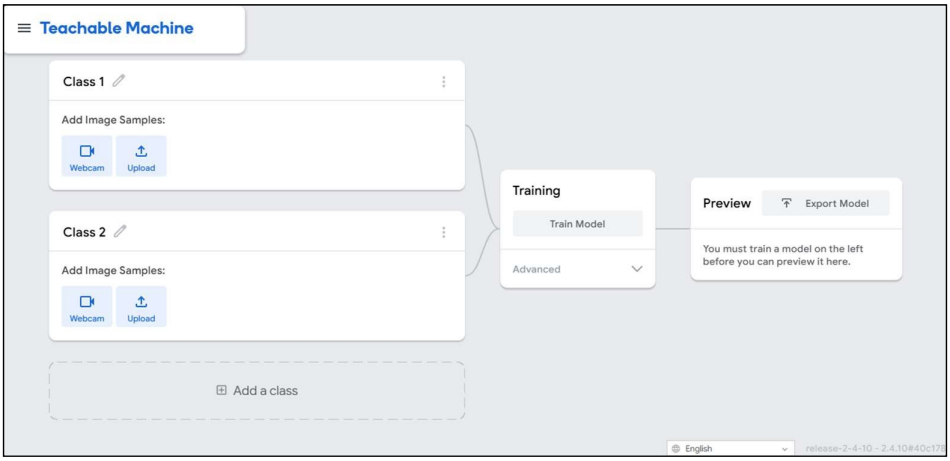
Step 3: Teachable machine website is open and click the Get Started button to start design the model.

Step 4: Choose the image project. There are two options, standard and embedded then choose standard.



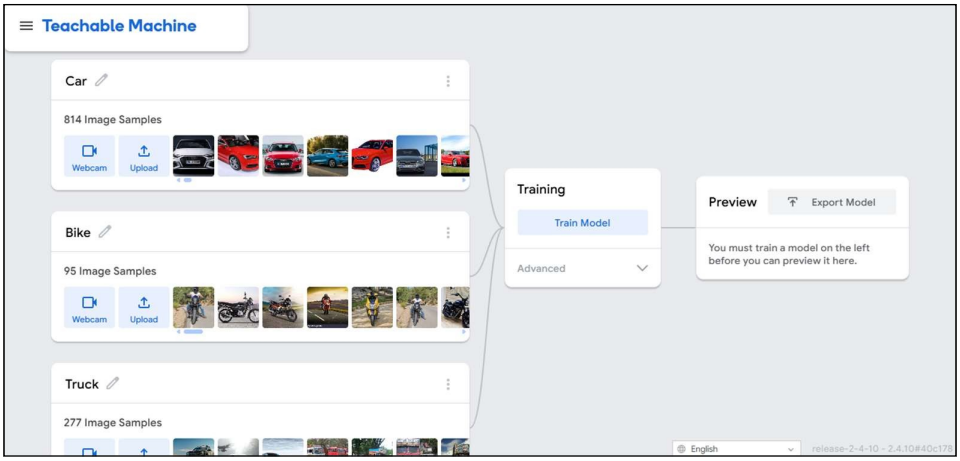


Step 5: After clicking on Standard Image Project, classes will be added to classify with the help of the model.

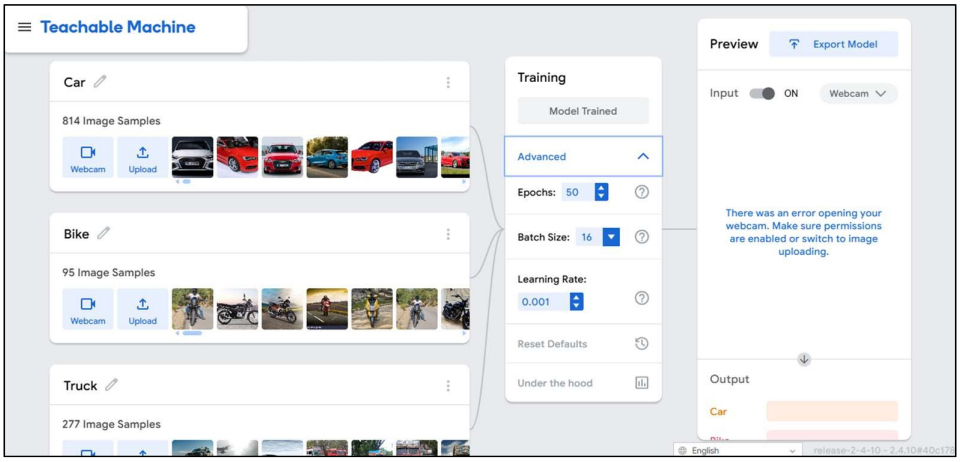


Step 6: Now, create classes and start uploading the images. Replace class1 as Car, class2 name as Bike and add another class3 name as Truck.

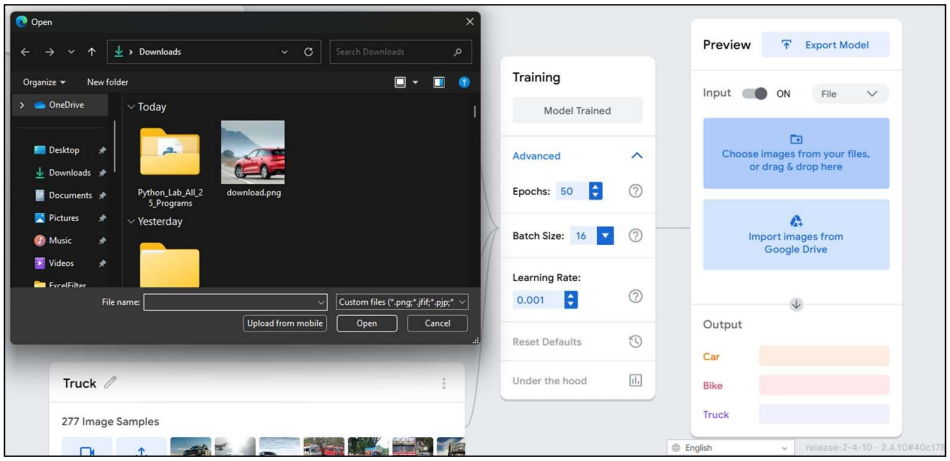
Step 7: Upload images from the dataset or use the live camera to capture the images.



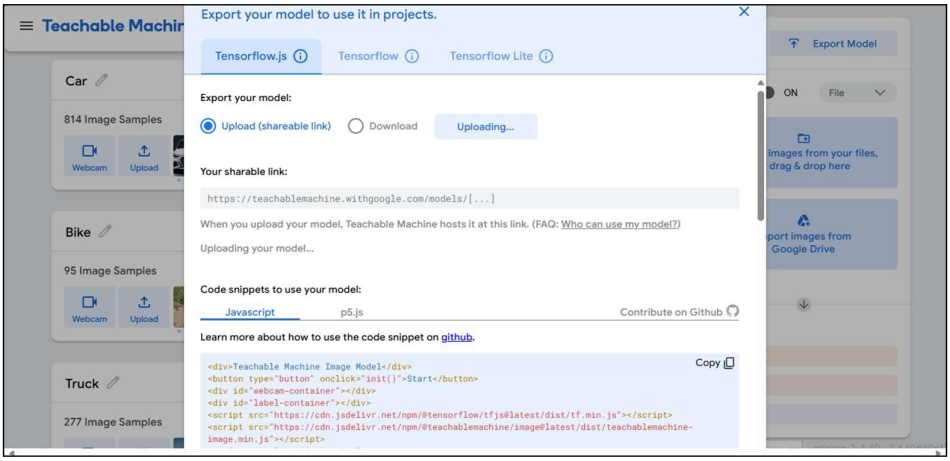
Step 8: After uploading the images, click on Train Model.



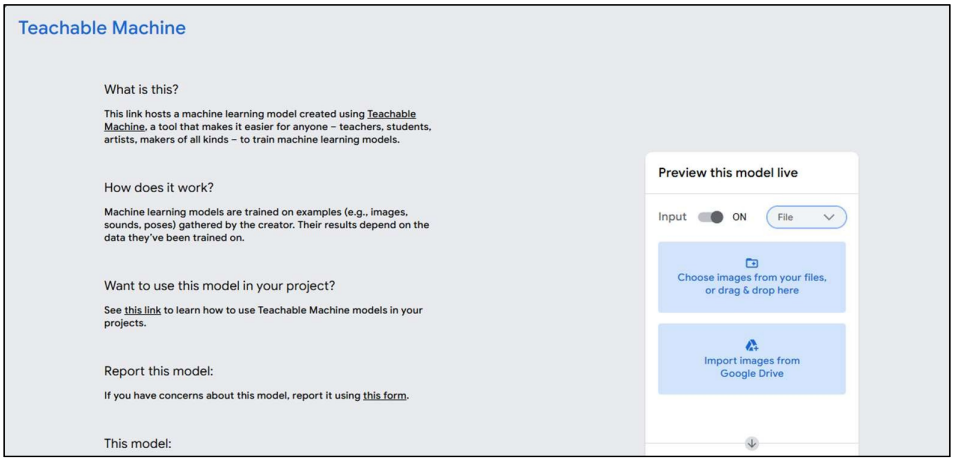
Step 9: Test the model by using the unseen dataset.



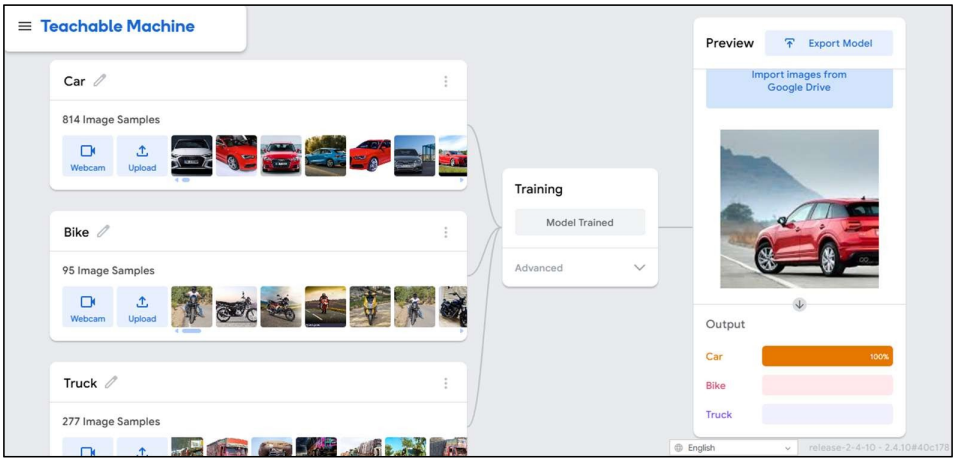
Step 10: After train and test the model, it's time to export the model. Click the uploaded my model and get the link, copy the link.

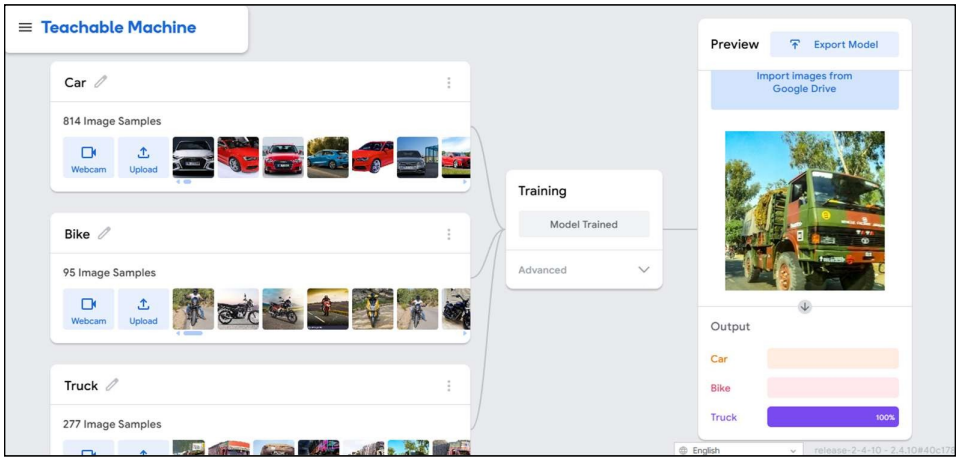


Step 11: https://teachablemachine.withgoogle.com/models/QLrTRFoCz/ paste the link in new tab.



Output:





Result:

Thus, the image classification model is trained and tested successfully using Teachable Machine.